

Social Data Science

Course Setup and Progression Log

Martin Davidsen

September 11, 2026

Purpose

This document serves as a personal setup and progression log for the Social Data Science course. It records the initial environment setup, Git and GitHub configuration, and the learning progress made as the course advances.

1 Session 1: Getting Set Up

1.1 Recommended environment

Technical explanation: The course recommends creating a dedicated Conda environment for Python work. This keeps the course packages separate from other Python projects on the computer. A typical setup is:

```
conda create -n bds python=3.11
conda activate bds
conda install pandas jupyter matplotlib
```

`conda create` Creates a new Conda environment named `bds` and installs Python 3.11 in it.

`conda activate` Switches the terminal into the `bds` environment so that its Python and packages are used.

`conda install` Installs the `pandas` data analysis library, Jupyter notebook tools, and `matplotlib` for plots in the active environment.

Plain-language explanation: This is like making one clean workbox for the course. Everything you need for this class lives in that box, so it does not mix with other projects. If you already created the `bds` environment earlier, you do not need to create a new one again. You only need to activate it.

The environment name `bds` is the course working environment used throughout this log.

1.2 Install Jupyter kernel in VS Code

Technical explanation: To ensure the notebook kernel appears in VS Code, install the IPython kernel in the environment:

```
conda activate bds
conda install ipykernel
# if needed as fallback
pip install ipykernel
python -m ipykernel install --user --name bds
```

`conda install ipykernel` Installs the kernel that allows Jupyter and VS Code to run Python code from the `bds` environment.

`pip install ipykernel` Provides the same installation as a fallback if Conda cannot install the package.

`python -m ipykernel` Registers the active environment as a selectable Jupyter kernel named `bds`.

Plain-language explanation: VS Code needs to know which Python setup to use for the notebooks. Without this step, the notebook may not show `bds` as an option. This step is usually done once. If the environment already exists and the kernel has been installed before, you usually only need to select `bds` in the notebook kernel menu.

After this, select the `bds` kernel in VS Code through the Jupyter kernel picker.

1.3 Useful reminders

- Use notebooks for exploration, analysis, and course exercises.
- Use Python scripts (files ending in `.py`) for reusable and structured workflows.
- Before trusting your results, restart the kernel and run all cells.
- Keep Markdown cells organized and descriptive.

Plain-language explanation: These reminders are simply good habits. Notebooks are helpful for trying things out, but they should still be run in the right order, saved properly, and checked before you trust the output.

1.4 How to work in VS Code

Technical explanation: The files ending in `.ipynb` are Jupyter notebooks, not ordinary terminal scripts. A notebook contains two kinds of cells:

- **Markdown cells** contain explanations, questions, instructions, and notes. They are not Python commands.
- **Python cells** contain code. They are run by the Python kernel selected in VS Code.

Plain-language explanation: Think of a notebook as a page with two different types of content: explanation sections and working code sections. The explanation sections are not code, while the code sections need a Python environment to run. In this course, you open a notebook, choose the `bds` kernel, and press play or use **Shift+Enter** to run the Python cells.

For this course, open a notebook in VS Code, select the `bds` kernel, and run Python cells with the play button or **Shift+Enter**. The terminal is used for commands such as `conda activate`, `git add`, and `git push`; it is not where notebook Python code normally belongs. For example, `import numpy as np` must be run in a Python cell. Typing it directly at a prompt ending in `%` makes `zsh` interpret `import` as a terminal command, which causes `command not found`.

1.4.1 A reliable notebook routine

1. Open the notebook in VS Code.
2. Select the `bds` kernel in the top-right kernel picker.
3. Run the first setup/import cell.
4. Continue from top to bottom so variables are created before they are used.

5. Read the Markdown cell before each exercise and attempt the task before opening the solution.
6. Save with `Cmd+S`.
7. Before submission, choose “Restart Kernel” and then “Run All” to check that the notebook works from a clean start.

Plain-language explanation: This is the safe way to work. You start with the setup section, run the notebook in order, and only then trust the results. This helps avoid strange errors caused by running cells out of order or forgetting to reload the environment.

2 Session 2: Git and GitHub

2.1 Git installation and identity

Technical explanation: Install Git if needed and configure your identity. I will use the name Martin Davidsen.

```
git --version
git config --global user.name "Martin Davidsen"
git config --global user.email "you@example.com"
```

`git -version` Checks that Git is installed and displays its version.

`git config` Stores the name Git will attach to your commits on this computer.

`-global`
`git config user.name` Stores the email Git will attach to your commits. Replace the example address
`-global` with your GitHub email.
`user.email`

Plain-language explanation: Git records who made the changes. The name and email are just like a signed label on your work. You should use the same name and email that you use on GitHub so your work is connected to your account.

Replace the email with the one you actively use for GitHub and course submissions.

2.2 Git basics

Technical explanation: These are the common commands used during the course:

```
git init
git status
git add file.py
git commit -m "message"
git log
```

`git init` Creates a local Git repository in the current folder.

`git status` Shows which files are new, modified, or staged for a commit.

`git add` Stages a file so it will be included in the next commit.

`git commit` Saves a named snapshot of the staged changes in the local repository.

`git log` Displays the history of commits.

Plain-language explanation: Git works like a version history for your project. It keeps track of what changed, when it changed, and what you decided to save. You do not need to use all the commands every time, but these are the most important ones to understand.

The usual workflow is:

```
Edit files -> git add -> git commit -> git push
```

In practice, this means editing files, staging the intended changes with `git add`, saving a local checkpoint with `git commit`, and uploading the result to GitHub with `git push`.

2.3 GitHub repository

Technical explanation: A repository for this course is already in place:

<https://github.com/MDCODE93/Introduction-to-Data-Handling>

A typical remote setup is:

```
cd '/Volumes/Samsung_T5/Uni2024/Social Data Science'
git init
git branch -M main
git remote add origin https://github.com/MDCODE93/Introduction-to-Data-Handling.git
git push -u origin main
```

`cd` Changes the terminal's current folder to the course folder.

`git init` Starts Git tracking in that folder. Run it only once for this local repository.

`git branch -M` Renames the current branch to `main`, the usual default branch name.

`main`

`git remote` Names the GitHub repository `origin` and connects it to the local repository.

`add origin`

`git push -u` Uploads the local `main` branch to GitHub and remembers this remote branch for future pushes.

The `git init` command is required when the local folder is not already a Git repository. If `origin` has already been added, use `git remote set-url origin <repo-url>` instead of adding it again.

2.4 Workshop notebook: commit and push

Technical explanation: The Session 2 workshop notebook is intended to be edited and pushed to the course repository. From the root of the local course folder, use:

```
cd '/Volumes/Samsung_T5/Uni2024/Social Data Science'
git add 'Introduction to Data Handling Eploration and Applied Machine Learning/ipynb/
workshop_notebook.ipynb'
git commit -m "Personalize workshop notebook"
git push -u origin main
```

`cd` Moves the terminal into the local course repository.

`git add` Selects only the workshop notebook for the next commit.

`git commit` Saves the selected notebook changes locally with a descriptive message.

`git push` Uploads the local `main` branch and notebook commit to GitHub. The `-u` option connects the local and remote branches for future pushes.

Plain-language explanation: This is just a small exercise to make sure you understand how notebooks and Git work together. You edit the file, save it, then upload the update to the online repository. It is best to add only the notebook you changed instead of uploading everything in the folder.

The notebook should be edited before this workflow, for example by changing `name = "Alex"` to `name = "Martin Davidsen"`. Adding the notebook by its full path avoids committing the entire course folder, generated files, or reading materials by accident.

3 Session 3: AI and Research Tools

3.1 Course focus

Technical explanation: Session 3 introduced tools for working more efficiently with AI and research workflows. The goal is to use digital tools responsibly while keeping the research process transparent, reproducible, and grounded in evidence.

Plain-language explanation: This session is about learning to work smarter with digital tools without losing your own understanding. AI can help explain ideas and suggest code, but you still need to check the result and understand why it makes sense.

3.2 Best practices

- Use AI as a support tool, not as a substitute for understanding the material.
- Keep track of prompts, outputs, and assumptions.
- Verify the correctness of generated code and explanations.
- Use version control for all assignments and project files.

4 Module 1 Notebook Walkthrough

This section explains the three Module 1 files currently used in the course. The first two are teaching notebooks. The third is an assignment template in which the same ideas are applied independently.

Plain-language explanation: This is the main teaching phase of the course. You first learn the idea in a small notebook, then you practice the same method on a larger assignment. The notebooks are not random tasks; they build on one another.

4.1 M1.01: From control flow to pandas

Technical explanation: Open `IntroductiontoDataHandlingExplorationandAppliedMachineLearning/ipython/M1_01_control_flow_to_pandas.ipynb` in VS Code and run it from top to bottom.

Plain-language explanation: This notebook is the first real bridge between coding and data analysis. It starts with simple rules and small examples, then moves to the table-based way of working that is central to this course.

Imports and versions	Imports <code>sys</code> , <code>NumPy</code> , and <code>pandas</code> , then prints the Python, <code>NumPy</code> , and <code>pandas</code> versions. This is a quick environment check.
Sample course records	Creates a small list of dictionaries. Each dictionary represents one illustrative course with a title, subject, price, and subscriber count.
Loops and conditions	Uses a <code>for</code> loop and <code>if/elif/else</code> to inspect one course at a time and assign a price category.

Functions	Places the price-category rules inside <code>price_tier</code> , so the same logic can be reused for many courses.
NumPy arrays	Converts prices and enrollments into arrays. <code>shape</code> describes the array dimensions and <code>dtype</code> describes the stored data type.
Boolean masks	Creates arrays of <code>True</code> and <code>False</code> values, then uses them to keep only courses meeting a condition.
DataFrame bridge	Converts the list of dictionaries into a pandas <code>DataFrame</code> , which is a labeled table with rows and named columns.
Historical catalog	Loads a larger teaching dataset from a local file when available, or from the stated repository URL otherwise. The cell reports which source it used.
Inspection	Uses <code>shape</code> , column names, <code>head</code> , <code>info</code> , and <code>describe</code> before analysis. These commands help you understand the table before making claims.
<code>.loc</code> and <code>.iloc</code>	Demonstrates the difference between selecting by labels and selecting by integer positions.
Pandas masks and assignment	Combines conditions with parentheses and <code>&</code> , selects rows with <code>.loc</code> , and creates an <code>investigate</code> column for matching rows.
<code>groupby</code>	Splits the catalog by subject, calculates totals, averages, and counts, and combines the results into a summary table.
Practice and solutions	Starter cells contain <code>None</code> or comments so you can attempt each task. The solution key is at the bottom; use it only after trying the exercise.

The main progression is as follows: explicit Python logic for one record, NumPy operations for arrays, and pandas operations for labeled tables. As the data structure becomes more powerful, the same question becomes both shorter and more scalable.

4.2 M1.02: Pandas deep dive

Technical explanation: Open `IntroductiontoDataHandlingEplorationandAppliedMachineLearning/ipynb/M1_02_pandas_deep_dive.ipynb` after M1.01. It uses a frozen 2020 Spotify playlist snapshot to compare energy and danceability across playlist genres.

Plain-language explanation: This notebook asks a more realistic question: how do we check whether the data are trustworthy before drawing conclusions? It teaches you to look carefully at the rows, the missing values, and the join logic before interpreting the numbers.

Load and provenance	Imports pandas, NumPy, and Matplotlib, then loads the class snapshot locally when present or downloads the stated URL. The dataset's date and source limit what can be concluded.
Row meaning	Establishes that a raw row records a track in a playlist with genre context. One track may occur in several playlists, so rows are not automatically unique tracks.
Standardise columns	Copies the raw table, cleans column names, and renames important fields such as artist, genre, and release date while preserving the raw frame.
Missingness audit	Counts missing values and creates a display label for unknown artists. A missing artist is different from a missing audio measurement and should not automatically be replaced with zero.

Date precision	Preserves release dates such as 2018, 2018-05, and 2018-05-23 with a year and a precision field. Converting every value to January 1 would invent precision that was not observed.
Join preparation	Checks whether repeated track IDs have conflicting attributes, creates a membership table and a one-row-per-track table, and defines the intended grain of each table.
Protected merge	Uses <code>validate='many_to_one'</code> to check that each membership finds at most one track record, <code>indicator=True</code> to show matches, and a row-count assertion to detect accidental duplication.
Conditional summaries	Filters high-energy associations, groups by genre, and calculates counts and mean danceability. The threshold is a chosen definition, not a universal fact.
Reshaping	Uses <code>pivot_table</code> for a compact comparison and <code>melt</code> to turn wide feature columns into a long table suitable for filtering and plotting.
Visualisation	Creates a horizontal bar chart while retaining association counts and the 0–1 feature scale. The chart describes the snapshot; it does not establish causation or current market behavior.

M1.02 emphasizes a more careful analysis workflow: define the unit of observation, inspect data quality, protect joins, summarize groups, reshape when useful, and explain the limits of the result.

4.3 M1 assignment: travelling tracks

The assignment starter is located at `IntroductiontoDataHandlingExplorationandAppliedMachineLearning/Assignments/M1_assignment_2026_starter.ipynb`. It is a separate notebook for group work, not a Python script. Replace the group number and member names in the first Markdown cell, then work through the notebook from top to bottom.

Setup cell	Loads pandas, NumPy, and Matplotlib, defines the two data URLs, and provides a reusable function that prefers local files before downloading from the URLs.
Question 1	Defines what one raw row and one working row represent, counts playlists per track, checks missing values, and derives a defensible release year.
Question 2	Predicts the row count before a merge, performs a validated merge, investigates key problems, and explains the cost of the chosen decisions.
Question 3	Compares travelling and non-travelling tracks by defining a threshold, checking sensitivity, comparing release years, reshaping the data, and creating one chart.
Interpretation	Explains the findings, the assumptions behind them, and two things the dataset cannot tell the curation team.
Decision log	Records one decision and its cost for each question so the analytical choices remain visible.
Submission check	Fill in the group information, complete the three questions, remove prompt comments, fill in the decision log, and use “Restart Kernel” followed by “Run All” before handing it in.

This assignment is where the course moves from copying notebook logic to explaining and defending the decisions behind the analysis.

5 Worked Solutions and Results

The following notes integrate the teaching notebooks in the `ipynb` folder with the assignment notebook in `Assignments`. They summarise the reasoning behind the solutions and the results observed when the notebooks were executed in the `bds` environment. The teaching data are downloaded from the URLs stated in the notebooks, so numerical results can change if the course source is updated.

5.1 Workshop notebook: Git and GitHub

The workshop notebook is a small practice exercise rather than a data analysis. Its purpose is to connect editing a notebook with the Git workflow.

The Python cell was solved as follows:

```
name = "Martin Davidsen"
print(f"Hello, {name}!")

for n in [1, 2, 3]:
    print(n)
```

The result was `Hello, Martin Davidsen!`, followed by 1, 2, and 3. The notebook was then staged by its specific path, committed, and pushed. This shows that a notebook is an ordinary file from Git's perspective, even though VS Code presents it as executable cells.

5.2 Solved M1.01: control flow, NumPy, and pandas

The first teaching notebook was executed successfully in the `bds` environment with Python 3.11.16, NumPy 2.4.6, and pandas 3.0.5.

5.2.1 Python record solutions

Technical explanation: The sample contains five course dictionaries. A loop over the records found **3,550** Web Development subscribers. The price function classified the two free courses as **Free** and the three paid courses as **Premium**. The price-times-enrollment values were 0, 240,000, 220,000, 0, and 75,000, for a total proxy of 535,000. This is explicitly only a proxy because price, enrollment, discounts, refunds, and fees are not actual revenue data.

Plain-language explanation: This first notebook is showing the logic behind a simple data check. The code is not trying to calculate real business revenue; it is just testing the pattern of the data and making sure the logic works before moving on to more advanced analysis.

The first practice solution collects titles rather than adding numbers:

```
free_titles = []
for course in sample_courses:
    if course["price"] == 0:
        free_titles.append(course["course_title"])
```

The result is `['Python Basics Lab', 'Security Lab']`. The important idea is that the loop checks one dictionary at a time and appends the requested field.

5.2.2 NumPy solutions

Technical explanation: The price array has shape `(5,)` and type `float64`; the enrollment array has shape `(5,)` and type `int64`. A Python list multiplied by 2 repeats its elements, while a NumPy array multiplied by 2 multiplies each element. The popular-enrollment mask is `[ True True False False]`.

Plain-language explanation: This section is about switching from working with individual values to working with whole lists of values at once. NumPy is faster and cleaner for this, and the mask is simply a yes/no filter that says which entries match a condition.

The second practice solution combines two element-wise comparisons. Parentheses are required because `&` combines boolean arrays:

```
subjects = np.array([course["subject"] for course in sample_courses])
web_and_popular = (subjects == "Web Development") & (enrollments >= 1000)
```

The result is `[ True False True False False]`. The mask keeps the first and third courses.

5.2.3 Pandas solutions and results

Technical explanation: The five records become a DataFrame with columns for title, subject, price, and subscribers. The historical catalog then loads 2,959 rows and 11 columns. The catalog has missing values in reviews, lectures, and content duration, but its core identifiers, subjects, prices, and subscriber counts are present.

Plain-language explanation: This is the moment where the course starts working with real tables. Instead of looking at one row at a time, it is now looking at a whole dataset and asking which rows match a condition and what patterns appear when the table is grouped.

The `.loc` demonstration selects by labels, while `.iloc` selects by integer positions; both return 20 in the tiny example only because the selected label happens to be at position 1.

The paid-and-popular practice solution is:

```
paid_and_popular = (df["is_paid"] == True) & (df["num_subscribers"] >= 100000)
display(df.loc[paid_and_popular,
               ["course_title", "price", "num_subscribers"]].head())
```

The solution finds two rows in the displayed result: *The Web Developer Bootcamp* and *The Complete Web Developer Course 2.0*. The reusable pattern is to build a boolean Series, then use it inside `.loc`.

The group summary reports total subscribers of 7,301,548 for Web Development, 1,738,412 for Business Finance, 907,807 for Graphic Design, and 779,127 for Musical Instruments. Web Development also has the highest average subscribers, 7,863.60, among Beginner Level courses. These are descriptive results from the historical catalog, not claims about today's market.

5.3 Solved M1.02: pandas deep dive

Technical explanation: After `matplotlib` was installed in `bds`, the second teaching notebook executed from start to finish. It loaded a 2020 Spotify playlist snapshot with 32,833 raw rows, 28,356 unique tracks, and 471 playlists.

Plain-language explanation: This notebook is about careful checking. Instead of jumping straight to the graph, it asks what the data actually represent, whether the rows are clean, and whether the merge is making sense before any conclusion is made.

5.3.1 Data quality and row grain

The raw data contain five missing titles, five missing artists, and no missing energy or danceability values. Release-date precision is preserved instead of invented: 30,947 rows contain a full day, 1,855 contain only a year, and 31 contain a year and month. There were no unrecognized non-missing date shapes.

The notebook creates a membership table with one row per recorded track-playlist-genre association and a track table with one row per track. It found 259 track-playlist pairs with multiple recorded genre labels. It also found no conflicting track attributes, so the one-row-per-track table is defensible for the audio features.

The protected merge produced 32,510 rows before and after joining. All 32,510 rows matched. The unchanged row count is important: it shows that the many-to-one join did not accidentally multiply the observations.

5.3.2 Grouped answer

The genre summary was:

Genre	Associations	Energy	Danceability	Energy coverage
edm	5,899	0.801	0.655	1.00
rock	4,866	0.733	0.520	1.00
latin	5,061	0.711	0.714	1.00
pop	5,507	0.701	0.639	1.00
rap	5,746	0.651	0.718	1.00
r&b	5,431	0.591	0.670	1.00

EDM has the highest mean energy, while rap has the highest mean danceability. At the threshold `energy >= 0.8`, EDM has the most high-energy associations (3,433), followed by rock (2,232). Lowering the threshold to 0.7 changes the selected counts, with EDM still highest at 4,659 and pop next at 3,115. The threshold is an analytical choice, not a natural law.

The long-format solution represents one genre–measure mean per row. The final chart orders genres by mean danceability while retaining both audio measures on the same 0–1 axis. The results describe playlist associations in the 2020 snapshot; repeated tracks and playlist context mean they are not independent observations of all music.

5.4 Solved M1 assignment: travelling tracks

Technical explanation: The assignment starter is intentionally blank. The solution below makes the row decisions explicit so the code can be explained rather than copied without understanding.

Plain-language explanation: This is the point where the course moves from learning the tools to using them properly. Instead of just copying code, you need to think about what each row means, which rows to include, and what assumptions you are making when you group or merge the data.

5.4.1 Before writing code: identify the tables

The first task is to decide what a row means at each stage of the analysis. The raw songs table is an association table: one row represents one recorded appearance of a track in a playlist, together with the playlist’s genre label and the track-level measurements supplied by the data source. Consequently, a popular track can appear many times. It is not correct to count raw rows as tracks or to calculate a track-level mean before deciding how repeated appearances should be handled.

The assignment uses three related tables. `songs_raw` retains the original playlist associations, `track_summary` has one row per unique `track_id`, and `family_associations` contains only valid track–family links after the family lookup has been checked. Giving each table a name that reflects its grain makes later code easier to audit: a `groupby` on `track_summary` answers a track-level question, while a count on `songs_raw` answers an association-level question.

The setup cell uses the notebook’s `load_class_csv` function. It checks several expected local paths and falls back to the URL when no local copy exists. This makes the notebook usable both with downloaded course files and from a clean machine. The two raw frames are loaded before any transformation:

```
songs_raw = load_class_csv("spotify_songs.csv", SONGS_URL)
families_raw = load_class_csv("subgenre_families.csv", FAMILIES_URL)
```

Plain-language explanation of the setup lines.

`songs_raw = ...` Opens the song file and stores the complete original table under the name `songs_raw`. The word “raw” reminds us not to overwrite this evidence while analysing it.

`families_raw =` Opens the lookup file that says which broad family each subgenre belongs to.

`load_class_csv()` tries the local course folders first. If the file is not there, it downloads the same file from the course URL.

`print(...shape..)` Reports how many rows and columns were loaded. This checks that the expected data arrived.

The setup also prints the number of rows and columns. This is an early reproducibility check: if a future download has a different shape, the later numerical results should not be accepted without investigation. Keeping the raw frames unchanged makes it clear that every later table is derived from the source data.

5.4.2 Question 1: define a track table

The raw file has 32,833 rows, 28,356 unique tracks, and 471 unique playlists. One row records a track in a playlist with genre and audio-feature context. For a track-level question, the working table should have one row per `track_id`; it adds `playlist_count`, the number of distinct playlists containing that track.

```
track_summary = (songs_raw.groupby("track_id", as_index=False)
                  .agg(track_name=("track_name", "first"),
                      track_popularity=("track_popularity", "first"),
                      release_date=("track_album_release_date", "first"),
                      playlist_count=("playlist_id", "nunique")))
track_summary["release_year"] = pd.to_numeric(
    track_summary["release_date"].astype("string").str[:4],
    errors="coerce")
```

Plain-language explanation of the track-table lines.

New table Gives a name to the new table. It will contain one summary card for each track.

Group by track Puts all records for the same track together. A track in five playlists is still one track, with five appearances.

Choose Says what single information to keep for each track.

summary Keeps the first recorded name because the repeated records agree here.

Keep the first values

name Keeps the popularity score once instead of repeating it for every playlist.

Keep the popularity

Keep the release date Keeps the recorded date text so its year can be used later.

Count different playlists Counts different playlists, so a duplicated row cannot falsely increase the travel count.

Take the first four characters	Treats the date as text and takes its year, such as 2019 from 2019-06-14.
Make the year numeric	Changes the extracted year into a number. Unreadable values are left blank rather than guessed.

The parentheses around the whole expression allow the method calls to continue over several lines. `groupby("track_id", as_index=False)` creates one group for every track and keeps the grouping key as an ordinary column in the result. The `agg` dictionary then specifies both the output names and the operation for each column. `first` keeps one representative track attribute because the snapshot contains consistent values for repeated IDs; `nunique` counts distinct playlists rather than raw rows. The latter distinction matters: if a source contained a duplicate copy of the same track-playlist association, `count` would overstate circulation but `nunique` would not.

The resulting table has one row per track, so `playlist_count` can be used to define travelling without giving tracks with many raw rows extra weight. The release-year code converts the date column to pandas' nullable string representation, takes the first four characters, and converts those characters to numbers. `errors="coerce"` turns malformed or missing values into `NaN`, which is preferable to inventing a year or stopping the whole analysis. The code does not turn a year-only date into a complete calendar date: it extracts only the precision actually present in the source.

Useful checks after this step are:

```
display(track_summary[["track_id", "playlist_count"]].head())
assert track_summary["track_id"].is_unique
assert track_summary["playlist_count"].ge(1).all()
print(track_summary["track_id"].nunique())
```

Plain-language explanation of the checks.

Show a few rows	Lets us see whether the result looks as intended.
Check unique IDs	Checks that no track ID appears twice in the summary.
Check playlist counts	Checks that every track has at least one playlist.
Count rows twice	Equal counts support the one-row-per-track claim.
Count empty values	Shows whether missing information is hidden in the columns we plan to use.
Count date lengths	Shows that dates do not all have the same precision.

The displayed rows make the working table visible. The first assertion confirms that the intended grain was achieved, while the second checks that every retained track appeared in at least one playlist. The number of unique IDs should equal the number of summary rows. The notebook also displays missingness in the variables used and counts the observed date-string lengths: 1,855 values have four characters, 31 have seven, and 30,947 have ten. This is evidence for preserving year precision rather than parsing every value as a full date.

Track attributes are consistent across repeated IDs in this snapshot. Five titles and five artists are missing, but popularity, playlist IDs, and release dates are available. The release year is derived from the observed year prefix; converting a year-only value to January 1 would invent a day.

5.4.3 Question 2: attach family labels safely

The family file has 24 rows but only 23 unique subgenres because `dance pop` maps to both `club_electronic` and `pop_craft`. The observed song data also contain `neo soul` and `tropical`, which are absent from the family file. A direct merge would therefore either duplicate rows for `dance pop` or leave some tracks without a family.

The conservative solution is to check the key, exclude ambiguous and unmapped family associations from family-level comparisons, and report that loss explicitly:

```
family_key_counts = families_raw.groupby("subgenre")["family"].nunique()
ambiguous_subgenres = set(family_key_counts[family_key_counts > 1].index)
family_one_to_one = families_raw[
    ~families_raw["subgenre"].isin(ambiguous_subgenres)
]

track_subgenres = songs_raw[["track_id", "playlist_subgenre"]].drop_duplicates()
track_subgenres = track_subgenres.rename(
    columns={"playlist_subgenre": "subgenre"})
eligible_associations = track_subgenres[
    track_subgenres["subgenre"].isin(family_one_to_one["subgenre"])
]
expected_rows = len(eligible_associations)
family_merge = track_subgenres.merge(
    family_one_to_one, on="subgenre", how="left",
    validate="many_to_one", indicator=True)
family_associations = family_merge.loc[
    family_merge["_merge"] == "both",
    ["track_id", "subgenre", "family"]
].copy()
```

Plain-language explanation of the family-lookup lines.

Count family labels	Checks whether each subgenre has one clear family or several competing answers.
List ambiguous labels	Finds subgenres with more than one possible family.
Keep clear labels	Sets ambiguous rows aside instead of guessing.
Remove duplicate pairings	Builds a list where each track-subgenre pairing appears once.
Use the same column name	Makes the subgenre column match in both tables.
Predict the row count	Finds the pairings with a safe lookup before the merge.
Join the tables	Places a family label beside each pairing while keeping unmatched cases visible.
Check the join	Stops if one subgenre unexpectedly points to several remaining families.
Mark each result	Adds a note saying whether the pairing matched.
Keep matched rows	Keeps only rows where both the track pairing and family label were found.

The first groupby is a key audit, not an analysis summary. `nunique` counts how many different family values each subgenre points to. A key with one value is a valid lookup key; a key with two values is ambiguous. Converting the ambiguous index to a `set` makes membership testing fast and keeps the filtering expression readable.

`drop_duplicates()` creates one row per observed track-subgenre association before the merge. Without it, repeated playlist appearances could be repeated again after the family lookup and family counts would partly measure playlist repetition instead of labelled tracks. The re-name makes the join key have the same name in both tables. The pre-merge filter calculates the expected number of valid associations. The left merge retains unmatched rows for inspection, `indicator=True` labels them as `both`, `left_only`, or `right_only`, and `validate="many_to_one"` asserts that each observed subgenre matches at most one family. If a future source introduced another duplicate family mapping, pandas would raise an error instead of silently multiplying rows.

For a complete audit, the notebook compares observed keys with the checked lookup keys and prints the ambiguous and unmapped values. It then keeps `family_associations` only where `family_merge["_merge"] == "both"`. The result is 28,610 valid associations and 4,223 unmatched associations. This distinguishes an ambiguous key, which is deliberately excluded, from an unmapped key, which was never present in the lookup table. Neither is silently assigned a family.

5.4.4 Question 3: define travelling and compare groups

I define a travelling track as a track appearing in at least three distinct playlists. This is a transparent threshold that selects repeated circulation rather than a single occurrence. The comparison code is:

```
track_summary["travelling"] = track_summary["playlist_count"] >= 3
comparison = track_summary.groupby("travelling").agg(
    tracks=("track_id", "size"),
    mean_playlists=("playlist_count", "mean"),
    mean_popularity=("track_popularity", "mean"),
    mean_release_year=("release_year", "mean"),
)
```

The comparison is performed on `track_summary`, not on the raw association table. First, the Boolean expression creates one value per unique track: `True` when the track appears in at least three distinct playlists and `False` otherwise. `groupby("travelling")` then creates the two comparison groups. In `agg`, `size` counts rows in each group, while the other named aggregations calculate means of the corresponding track-level variables. Reporting `tracks` beside every mean is essential: a mean from a thin group should not be presented with the same confidence as a mean based on thousands of tracks.

There are 705 travelling tracks and 27,651 non-travelling tracks. Travelling tracks average 3.53 playlists and popularity 71.38; non-travelling tracks average 1.08 playlists and popularity 38.51. Their mean release years are approximately 2012.05 and 2011.03 respectively. The popularity difference is descriptive and may reflect selection, recency, or curation; it does not prove that travelling causes popularity.

At a lower threshold of at least two playlists, there are 2,818 travelling tracks with mean popularity 59.54. At a stricter threshold of at least four playlists, there are 224 travelling tracks with mean popularity 78.58. The conclusion depends on the cut-off, which is why the assignment asks for sensitivity analysis.

The threshold sensitivity is implemented by changing only the Boolean rule and rerunning the same summary. That is preferable to copying the analysis into several separate code blocks because the comparison remains structurally identical:

```
for threshold in [2, 3, 4]:
    travelling = track_summary["playlist_count"] >= threshold
    print(threshold, travelling.sum(),
          track_summary.loc[travelling, "track_popularity"].mean())
```

Here `travelling.sum()` works because pandas treats `True` as 1 and `False` as 0. The filtered `.loc` expression selects only tracks meeting the current threshold. The output shows whether the substantive pattern is stable or whether it is being driven by a small, highly repeated group. A stricter threshold changes both the definition and the sample composition, so the results should be described as sensitivity checks rather than as three independent discoveries.

Plain-language explanation of the travelling-track lines.

Mark travelling Adds a yes/no label for tracks appearing in at least three playlists.
tracks
Make two Separates the yes and no tracks and calculates the same summaries for both.
groups
Count tracks Counts tracks in each group, so every average has a sample size beside it.

Average Calculates the average number of playlists for each group.
playlist count
Average Calculates the average popularity score for each group.
popularity
Average release Helps us consider whether newer tracks explain part of the difference.
year
Test three Repeats the comparison with thresholds of two, three, and four playlists.
thresholds
Make the Creates the yes/no decision for the chosen threshold.
current rule
Count yes Counts the tracks that meet the current definition.
values
Select Keeps only tracks meeting the current definition before averaging popularity.
matching
tracks The family stage then attaches the track-level summary to the valid family associations:

```
family_track_data = family_associations.merge(
    track_summary, on="track_id", how="left", validate="many_to_one")
family_track_unique = family_track_data.drop_duplicates(["family", "track_id"])
family_comparison = family_track_unique.groupby("family").agg(
    tracks=("track_id", "nunique"),
    travelling_tracks=("travelling", "sum"),
    mean_popularity=("track_popularity", "mean"),
)
family_comparison["travelling_share"] = (
    family_comparison["travelling_tracks"] / family_comparison["tracks"])
```

Plain-language explanation of the family-comparison lines.

Add track Places the travelling label beside the family links already checked.
information
Check the Confirms that each track finds only one matching summary row.
second join
Remove Makes sure a track is counted only once inside a family.
duplicate
Summarise Calculates its track count, travelling-track count, and average popularity.
family links
each family
Count Counts yes labels as travelling tracks.
travelling
Calculate the Divides travelling tracks by all tracks in the family.
tracks
share

The second merge is many-to-one because several family associations may refer to the same track, but each track ID has only one row in `track_summary`. The additional `drop_duplicates` prevents a track linked to the same family through more than one subgenre from being counted twice. In this output, the smallest family has 2,045 valid tracks, so no family falls below the notebook's 100-track warning threshold. The chart uses `sort_values` and a horizontal bar plot to make the travelling-track counts easy to compare.

Finally, the notebook reshapes selected family measures with `melt`. Wide columns such as `tracks`, `travelling_tracks`, and `mean_popularity` become rows under `measure` and `value`. Long form is useful when several measures need the same filtering or plotting workflow; the chart itself focuses on travelling-track counts. These transformations preserve the descriptive nature of the analysis: they organise the evidence but do not create a causal design.

For the one-to-one family associations, the travelling-track counts are 353 for `hiphop_lineage`, 341 for `pop_craft`, 304 for `club_electronic`, 252 for `soul_rnb`, 204 for `latin_rhythm`, and 104 for `guitar_rock`. Family counts are based on valid labelled associations, so they should not be read as a complete classification of every track. A better future grouping would use a documented, one-to-one taxonomy that resolves overlapping and missing subgenres before the merge.

The assignment's final interpretation should state what the data can support: a descriptive comparison of playlist circulation and popularity in this snapshot. It should also state what it cannot support: causal claims about popularity, generalization to all music or current charts, or certainty about tracks whose family label is ambiguous or missing. The decision log records these choices and their costs.

6 Course Progression Log

Plain-language explanation of the final display steps. The notebook also changes the family table into a form that is easier to inspect and draws a chart. `reset_index()` turns the family names back into a normal column. `melt(...)` gathers the separate measures into two columns: one says what is being measured and the other gives its value. This is useful when several measures need to be compared in the same format. `sort_values("travelling_tracks")` orders the families from fewer to more travelling tracks. `plot(kind="barh", ...)` draws horizontal bars whose lengths show the counts. `tight_layout()` makes room for the labels, and `show()` displays the finished chart.

This section is intended to be updated throughout the course as skills and understanding improve.

6.1 Week 1: Initial setup completed

- Installed/verified the core tools required for the course.
- Created a dedicated Python environment named `bds`.
- Installed Jupyter and IPython kernel support.
- Configured Git with the user name **Martin Davidsen**.
- Connected to the GitHub repository: <https://github.com/MDCODE93/Introduction-to-Data-Handling>

6.2 Week 2: Git and GitHub workshop completed

- Opened the Session 2 workshop notebook in VS Code.
- Changed the example name from `Alex` to `Martin Davidsen`.

- Ran the Python cell successfully and verified the output: `Hello, Martin Davidsen!` followed by the numbers 1, 2, and 3.
- Added only `workshop_notebook.ipynb` to Git.
- Committed the change with the message `Personalize workshop notebook`.
- Pushed the commit to the `main` branch of the GitHub repository.

6.3 Week 3: Progress update

Add notes here about new tools, assignments, and improvements in workflow.

6.4 Week 4: Progress update

Add notes here about project work, analysis, and reflections.

7 Checklist for Future Work

- Keep the `bds` environment activated during coursework.
- Commit work regularly to Git.
- Push updates to the GitHub repository before deadlines.
- Save notebook versions and explain key steps in Markdown cells.
- Review the progression log after each session.